

Respostas — Engenharia e Gestão de Serviços

Respostas baseadas nos slides da cadeira. Em cada resposta, o **ponto-chave** está a negrito.

1. Dimensionamento de uma infraestrutura de Cloud Computing

Ao dimensionar uma infraestrutura de cloud, é preciso equilibrar várias características:

- **Carga (procura):** a carga varia muito e o pico pode ser 2 a 10 vezes superior à média (fins de semana, feriados, Black Friday, conteúdo viral).
- **Custo:** o hardware é caro de comprar (CapEx) e de manter (OpEx).
- **Escalabilidade:** tem de crescer e diminuir consoante a procura, mas escalar é lento e difícil.
- **Disponibilidade:** o sistema tem de estar quase sempre a funcionar (ex.: 99,999% = ~5 min de falha por ano).

Não existe solução única porque a carga é dinâmica e imprevisível, o que obriga sempre a um compromisso: ou se dimensiona para o pico (e desperdiçam-se recursos, com utilização típica de apenas 5%-20%), ou se dimensiona abaixo do pico (e perdem-se clientes por falta de capacidade). Não há um valor "certo" que sirva para todos os momentos.

2. Containers numa rede privada isolada e exposição no Kubernetes

Os containers estão numa rede privada isolada porque usam *network namespaces*, um mecanismo do Linux que isola e virtualiza os recursos de rede (interfaces, tabelas de encaminhamento). Cada namespace "vê" apenas os seus próprios recursos, como se fosse o host inteiro, e uma interface de rede só pode pertencer a um namespace. Assim, os processos dentro do container não veem as interfaces de rede do anfitrião.

Não se atribui uma interface real do host ao container (isso teria impacto no anfitrião). Em vez disso, cria-se um **par de interfaces virtuais (veth)** ligadas ponto-a-ponto: uma fica no namespace do container e a outra é ligada a uma *bridge* de software.

No **Kubernetes**, expõe-se um serviço à internet em dois passos:

- **Service:** dá um ponto de acesso estável (IP e nome DNS fixos) para alcançar os pods **dentro do cluster** (permite pods falarem com pods, com balanceamento round-robin). É interno.
 - **Ingress:** é o que torna o serviço **visível para o exterior (internet)**, normalmente com a ajuda de um *proxy* como o Nginx ou o Traefik.
-

3. Quatro princípios da Orientação a Serviços (SOA)

Os princípios da orientação a serviços definem como desenhar serviços independentes e reutilizáveis. Quatro deles:

1. **Loose coupling (acoplamento fraco)**: os serviços minimizam dependências entre si; só precisam de ter consciência uns dos outros.
2. **Service contract (contrato de serviço)**: os serviços seguem um acordo de comunicação, definido pelas descrições do serviço.
3. **Autonomy (autonomia)**: cada serviço controla a lógica que encapsula.
4. **Abstraction (abstração)**: para além do que está no contrato, o serviço esconde a sua lógica do exterior.

(Os outros quatro princípios são: **Reusability** — reutilização; **Composability** — poderem ser combinados; **Statelessness** — não guardarem estado; **Discoverability** — poderem ser descobertos.)

4. SAML Assertion

Uma SAML Assertion é uma declaração de factos sobre um sujeito (utilizador), feita por um Identity Provider (a "autoridade" que afirma quem o utilizador é). O Service Provider confia nessa declaração.

Uma assertion pode conter:

- factos de **autenticação** (o utilizador autenticou-se desta forma, a esta hora);
- **atributos** do sujeito (direitos de acesso, limites, estado);
- **decisões de autorização** já tomadas.

Para que serve: permite que dois servidores partilhem informação de identidade e autenticação, sendo a base do **Single Sign-On (SSO)** — o utilizador autentica-se uma vez e acede a vários serviços.

Vantagens:

- é um standard da indústria, **neutro** (não depende de um fornecedor);
 - baseado em **XML**;
 - qualquer ponto da rede pode afirmar que conhece a identidade (não depende de uma autoridade central de certificados);
 - suporta **identidade federada** (autenticação partilhada entre organizações diferentes).
-

5. Padrão Publish/Subscribe

No Publish/Subscribe, um serviço publica mensagens/eventos de um dado tipo (tópico) e os clientes subscrevem os tipos que lhes interessam; quando um evento é publicado, o sistema consulta a tabela de subscrições e reencaminha-o para os clientes interessados (normalmente colocando-o numa fila). A grande diferença é que o publicador e o subscritor não precisam de se conhecer.

Pode ser comparado ao **modelo por filas / ponto-a-ponto (point-to-point)**. A diferença: no ponto-a-ponto o cliente e o servidor conhecem-se (através de uma fila ou interface); no pub/sub a ligação é feita através de tópicos e eventos. Assemelha-se também a um **multicast** (uma publicação chega a muitos subscritores).

Vantagens:

- **desacoplamento** (publicadores e subscritores independentes uns dos outros);
 - comunicação **assíncrona** (não é preciso esperar por resposta);
 - **escalabilidade** (suporta facilmente um número variável de serviços);
 - maior **tolerância a falhas**;
 - um publicador consegue chegar a muitos subscritores.
-

6. Orquestração vs. Composição de serviços

Composição de serviços é o **juntar de vários serviços para automatizar uma tarefa ou processo de negócio**. Para haver composição, tem de existir pelo menos **dois serviços mais um iniciador**; caso contrário é apenas uma troca ponto-a-ponto.

Orquestração estabelece um **protocolo de negócio que define formalmente um processo, centralizando e controlando** a lógica entre aplicações através de um modelo padronizado. A lógica de *workflow* é dividida em atividades organizadas em sequências e fluxos.

A **distinção-chave**: a **composição** é apenas a **agregação de serviços**, enquanto a **orquestração** acrescenta um **controlo central que coordena esse processo** (existe um "maestro" que comanda tudo). Isto contrasta com a *coreografia*, em que não há controlo central e os participantes se coordenam diretamente entre si.

7. Principais componentes do 3GPP/IMS

O núcleo (core) do IMS é composto pela CSCF (Call Session Control Function) e pelo HSS (Home Subscriber Server). A CSCF divide-se em três tipos:

- **HSS (Home Subscriber Server):** a base de dados mestre com toda a informação dos subscritores. Trata da identificação, autenticação, autorização de acesso, gestão de mobilidade e suporte ao estabelecimento e provisionamento de serviços.
- **P-CSCF (Proxy-CSCF):** o **ponto de contacto do utilizador** para a sinalização SIP. Fica no domínio visitado; comprime e protege as mensagens SIP.
- **S-CSCF (Serving-CSCF):** **controla a sessão SIP do utilizador**. Fica no domínio de origem (home) e funciona como *registrar* SIP.
- **I-CSCF (Interrogating-CSCF):** o **ponto de contacto do domínio para a sinalização SIP entre domínios**; descobre qual o S-CSCF que serve o utilizador.

Componentes de apoio importantes:

- **MGCF + MGW (Media Gateway Control Function e Media Gateway):** fazem a ligação com a rede telefónica tradicional (GSTN).
 - **Application Servers (AS):** fornecem os serviços propriamente ditos (não fazem parte do core IMS) e interagem com o S-CSCF via SIP.
-

8. Modelo cliente-servidor vs. modelo por filas (reliable queueing)

O modelo cliente-servidor é síncrono e bloqueante; o modelo por filas é assíncrono e desacoplado, com mensagens persistentes.

Cliente-servidor (síncrono, pedido/resposta):

- o cliente faz o pedido e **fica bloqueado à espera** da resposta;
- é simples e intuitivo, parecido com a programação procedural e bem suportado por RPC;
- **desvantagens:** ambos têm de estar online ao mesmo tempo; mantém estado no cliente (consome CPU/RAM → problemas de escalabilidade); tem *overhead* de comunicação; se uma chamada falha, pode provocar uma cascata de falhas.

Modelo por filas (assíncrono):

- a comunicação é feita através de **filas persistentes que guardam as mensagens de forma transaccional** (as mensagens continuam lá mesmo depois de uma falha);
 - **as aplicações não precisam de estar disponíveis no momento em que o pedido é feito;**
 - maior **tolerância a falhas** e flexibilidade;
 - permite padrões complexos: 1-para-1, 1-para-muitos (multicast, útil para subscrições), muitos-para-1 e muitos-para-muitos;
 - **custo:** mais flexibilidade à custa de algum desempenho.
-

9. SAML vs. OAuth

SAML serve sobretudo para autenticação e Single Sign-On empresarial (baseado em XML); OAuth serve para autorizar o acesso a APIs (baseado em tokens).

SAML:

- standard OASIS (2005);
- baseado em **XML** (usa SOAP, XMLDSig, XMLEnc);
- foco em **autenticação e SSO**, incluindo identidade federada entre organizações;
- é considerado "enterprise-ready", mas é **pesado**;
- usa SSL entre servidores.

OAuth:

- standard aberto para **autenticação/autorização segura de APIs**;
 - iniciado pelo Twitter (2006);
 - baseado em **tokens** (o utilizador autenticado tem um token único para aceder aos dados);
 - mais **leve**; o OAuth1 não é extensível nem "enterprise-ready";
 - o **OAuth2** (2010) é um protocolo novo, valida por HTTPS e traz melhorias: *scoping* (direitos diferentes) e possibilidade de **revogar** direitos já concedidos.
-

10. Enterprise Service Bus (ESB)

Um Enterprise Service Bus (ESB) é uma arquitetura de software para um middleware (um "barramento" central) que fornece serviços fundamentais para arquiteturas mais complexas e que serve para implementar SOA. O ESB gere o acesso às aplicações e serviços, apresentando uma interface **única, simples e consistente**.

Funcionalidades principais:

- distribui a informação pela organização de forma rápida;
- **mascara as diferenças** entre plataformas, arquiteturas e protocolos de rede;
- garante a entrega da informação mesmo quando alguns sistemas estão offline;
- faz *routing*, registo (log) e enriquecimento sem obrigar a reescrever as aplicações;
- permite implementação **incremental** (não é preciso mudar tudo de uma vez).

Componentes: Mediator, Service Registry, Choreographer e Rules Engine.

11. Paradigmas do Cloud Computing (IaaS, PaaS, SaaS)

Os três paradigmas são camadas de serviço que vão abstraindo cada vez mais a infraestrutura: IaaS → PaaS → SaaS.

- **IaaS (Infrastructure as a Service):** aluga-se **infraestrutura** virtualizada (servidores, armazenamento, redes). Dá acesso à stack de infraestrutura: SO completo, firewalls, routers, load balancing. Ex.: Amazon EC2/S3, Google Cloud, IBM Bluemix.
- **PaaS (Platform as a Service):** aluga-se uma **plataforma** de desenvolvimento. O programador foca-se só em criar a aplicação, sem se preocupar com o SO, escalabilidade ou load balancing. Ex.: Google App Engine, MS Azure, Heroku.
- **SaaS (Software as a Service):** aluga-se o **software** já pronto, acedido pelo browser/web services. Não é preciso instalar nada, os upgrades são automáticos e acede-se aos dados em qualquer lado. Ex.: Google Apps, Salesforce, Office 365.

Vantagens comuns: pagar pelo uso (*pay-per-use*), escalabilidade instantânea, segurança e fiabilidade.

12. Public cloud vs. Private cloud

A diferença principal está em quem pode usar: a cloud pública é aberta e comercial, a cloud privada é restrita a uma única organização.

- **Public cloud:** serviço comercial, aberto a (quase) qualquer pessoa. Ex.: Amazon AWS, Microsoft Azure, Google App Engine.
- **Private cloud:** partilhada apenas **dentro de uma única organização**. Ex.: o datacenter interno de uma grande empresa.

(Existe ainda a **Community cloud**, partilhada por várias organizações semelhantes — ex.: a "Gov Cloud" da Google.)

13. Como um Service Center (SC) envia um SMS para um MT

Nota: este tema não aparece nos slides fornecidos. Resposta com base no funcionamento standard do SMS.

O Service Center (SMSC) funciona em modo "store-and-forward": guarda a mensagem e depois entrega-a, consultando primeiro o HLR para localizar o destinatário. Passos para entregar a um MT (Mobile Terminated):

1. O SMSC **recebe e armazena** a mensagem.
 2. O SMSC interroga o **HLR (Home Location Register)** para saber em que MSC/rede o destinatário está registado.
 3. O HLR responde com o endereço do **MSC** que serve o utilizador.
 4. O SMSC envia a mensagem para esse MSC (por sinalização SS7/MAP).
 5. O MSC obtém a informação do subscritor no **VLR** e faz o *paging* do telemóvel.
 6. A mensagem é **entregue ao terminal** e é enviada uma confirmação de entrega de volta ao SMSC.
 7. Se o terminal estiver indisponível, o SMSC **guarda a mensagem e volta a tentar** mais tarde.
-

14. Como é enviado um MMS

Nota: este tema não aparece nos slides fornecidos. Resposta com base no funcionamento standard do MMS.

O MMS usa um centro central (MMSC) que guarda a mensagem e notifica o destinatário, que depois descarrega o conteúdo — ou seja, mistura rede de dados (HTTP/WAP) com uma notificação por SMS. Passos:

1. O remetente envia o MMS para o **MMSC (MMS Center)**, tipicamente por HTTP/WAP sobre a rede de dados.
 2. O MMSC **armazena** a mensagem.
 3. O MMSC envia uma **notificação (WAP Push, sobre SMS)** ao destinatário a avisar que há um MMS à espera.
 4. O terminal do destinatário liga-se ao MMSC e faz o **download** do conteúdo (por HTTP/WAP GET).
 5. O MMSC envia/recebe as confirmações.
 6. Se a mensagem for para outra operadora, o MMSC **reencaminha-a** para o MMSC da rede de destino.
-

15. O que é um RCS

Nota: este tema não aparece nos slides fornecidos. Resposta com base no conhecimento standard.

RCS (Rich Communication Services) é a evolução do SMS/MMS: um standard da GSMA para mensagens "ricas" que funciona sobre IP. Ao contrário do SMS/MMS, oferece funcionalidades semelhantes às apps de mensagens modernas, integradas na aplicação de mensagens da operadora:

- mensagens de **grupo**;
- envio de **imagens e vídeos** em alta resolução e partilha de ficheiros;
- indicadores de "**a escrever...**" e **recibos de leitura**;
- **presença** (saber se o contacto está disponível);
- chamadas de voz/vídeo.

Em resumo, o RCS pretende ser o substituto nativo do SMS e do MMS, com funcionalidades modernas, usando IP em vez da rede de sinalização tradicional.

16. Modelo cliente-servidor vs. modelo por filas (reliable queueing)

┃ Esta pergunta é igual à pergunta 8. Ver a resposta detalhada acima.

Resumo: o **cliente-servidor** é **síncrono e bloqueante** (o cliente espera pela resposta, ambos têm de estar online, mantém estado e tem problemas de escalabilidade). O **modelo por filas** é **assíncrono e desacoplado**, com **mensagens persistentes** guardadas de forma transacional (as aplicações não precisam de estar disponíveis no momento do pedido), o que dá mais tolerância a falhas e flexibilidade, mas com algum custo de desempenho.